Introduction

The n-body problem studies how multiple objects move under mutual gravitational interaction. Despite its simple formulation, its behavior quickly becomes complex due to the quadratic number of pairwise interactions, making it computationally expensive for large systems.

This project aims to implement an efficient simulation of the n-body problem using classical numerical integration methods, specifically Velocity Verlet, and to explore performance optimizations through vectorization with NumPy and parallel computation using CUDA. In addition to the physical model, the project also focuses on software design aspects such as modular backends, controllable simulation loops, and reproducible environments.

The goal is to balance physical plausibility, numerical stability, and computational performance while maintaining a flexible architecture for future extensions.

Physics

The branch of physics that we care about for this project are the Isaac Newton's classical mechanics and its laws of motion, I believe.

When I had to learn this at school, Wikipedia [1] and CrashCourse's YouTube's playlist on physics [2] made me understand mechanics.

Let's try to summarize the basics.

Kinematic Equations

Every object has its *position*, like it's coordinates, represented by r.

When that object is in motion, there is a *velocity*, v, that is, the *speed* of the object in a certain *direction*. When there's no motion, $v = 0$. This speed and velocity are normally represented as the magnitude and direction — in Portuguese *sentido*, since *direção* alone doesn't let us represent the inverse of a vector — of a vector respectively.

Given a constant velocity (no acceleration) v, and Δt being the time passed, starting at t_0 and ending at t_f, $r(\Delta t) = r(t_0) + vt_f$[1], so r is the position when the object moves with that velocity v for t units of time starting at position $r(t_0)$.

The average velocity is $\bar{v} = \frac{\Delta r}{\Delta t}$.[2] When we close that time window to a certain point of time, we tend to get the velocity at that point of time.[3] The velocity at given time t (instantaneous velocity) is represented as:

$$
\begin{aligned}
v(t) &= \lim_{\Delta t \to 0} \frac{\Delta r}{\Delta t} = \\
&= \lim_{\Delta t \to 0} \frac{r(t + \Delta t) - r(t)}{\Delta t} = \\
&= \frac{dr}{dt} = r'(t)
\end{aligned}
$$

Velocity changes position over time, and acceleration changes velocity over time.

So something similar happens to calculate the acceleration a:

---

[1] [3]
[2] [4]
[3] [5]

$$
\begin{aligned}
a(t) = \lim_{\Delta t \to 0} \frac{\Delta v}{\Delta t} = \\
= \frac{dv}{dt} = v'(t) = \\
= \frac{d^2 r}{dt^2} = r''(t)
\end{aligned}
$$

Because we don't like in a unidimensional world, v and a are represented in vectors:

$$
\begin{aligned}
\vec{a} = \frac{\Delta \vec{v}}{t} \Leftrightarrow \vec{a}t = \Delta \vec{v} \Leftrightarrow \\
\Leftrightarrow \vec{v} = \vec{v_0} + \vec{a}t
\end{aligned}
$$

We can also use integrals to arrive to some common equations:[4]

$$
\begin{aligned}
\overrightarrow{\Delta r} = \int_{t_0}^{t} \vec{v} dt = \\
= \vec{v_0}t + \frac{\Delta \vec{v} t}{2} = \\
= \vec{v_0}t + \frac{\vec{a}t^2}{2}
\end{aligned}
$$

That was assuming the acceleration is constant. From that, we arrive to the kinematic equation[5]

$$
r = r_0 + \vec{v_0}t + \frac{1}{2}\vec{a}t^2
$$

For Later

There are some equations that appear on the project statement that we can talk about right now.

We can calculate the new position of an object if we have a record of the of it's last its position, velocity and acceleration at a certain point of time. We then know how much time has passed, so from the last equation we get:

$$
r(t + \Delta t) = r(t) + v(t)\Delta t + \frac{1}{2}a(t)\Delta t^2
$$

We can do the same for the velocity:

$$
v(t + \Delta t) = v(t) + a(t)\Delta t
$$

On the project statement, the teacher suggests a *half-step* and finally a correction. I still have to figure out why exactly, but the logic is the same.

Newton's Laws of Motion

1. For the velocity of an object to change, there must be a force acting upon it;
2. The linear momentum $\vec{p} = m\vec{v}$ of the object will change depending on that force applied. All the forces acting upon the object at the moment, together, equal how much the linear momentum is changing at that time.

   That is, assuming mass doesn't change over time:

[4] [6]
[5] [7]

$$\vec{F} = mv'(t) = m\vec{a}$$

3. If you hit a wall, the wall hits you back with equal force (and that's why it hurts).

   So for any force, there's an equal one but with inverse *sentido*.

Newton's Law of Universal Gravitation

This part about the history of the apple falling on Newton's head, we are learning together! Or at least I'll relearn something that didn't get stuck in my head this last year's I had physics in school.

> Newton's law of universal gravitation describes gravity as a force by stating that every particle attracts every other particle in the universe with a force that is proportional to the product of their masses and inversely proportional to the square of the distance between their centers of mass.
>
> — [8]

That gives the following:

$$F_{\text{gravity}} \propto \frac{m_a m_b}{\|\vec{r_a} - \vec{r_b}\|^2}$$

With empiricism, Newton figured out that the bigger the masses of the objects, the stronger the force; and the further apart the objects are from each other, the weaker the force.

Well, one thing is for sure: I am not some sort of magnet that when I'm walking around things just come towards me, and I only walk into a streetlight by the force of distraction, not by some sort of gravitational force. At the same time, when I fall, I find the ground and not the sky. Right. Newton came to the conclusion that the force between me and the Earth's surface (between me and the Earth really) is much, much greater than the force between me and… Imagine something really, really, big, and probably heavy, like a whale, or a huge building, the biggest in the world. That mass is nothing compared to the Earth's mass. But certainly there are heavier things than the Earth, right? Probably the Sun, for example. Yeah, probably, but don't forget that we are touching the Earth, and **the Sun is probably an astronomical unit away**, and that the distance between the objects also count.

Because of all the above, Newton added to the equation a constant G which represents just a very little and tiny number that would make those force calculation results seem more realistic. G initially didn't have a specific value attributed to it by Newton. Later scientists figured out that a good number for it is $G = 6.6743015 \times 10^{-11} \text{m}^3\text{kg}^{-1}\text{s}^{-2}$.[6][7] As Newton figured out, pretty tiny. If we say $\overrightarrow{\Delta r_{\text{ab}}}$ is the distance from object a to object b, the formula becomes:

$$\overrightarrow{F_{\text{gravity}}} = G\frac{m_a m_b}{\left\|\overrightarrow{\Delta r}\right\|^2}\frac{\overrightarrow{\Delta r}}{\left\|\overrightarrow{\Delta r}\right\|}$$

We get a force vector. We can also notice that the gravitational force from b to a is equal to that from a to b, but just the inverse vector (the third law of motion). Look, how cool is this! If you are free-falling, the force of gravity acting on you, based on the second law of motion, is:

$$\overrightarrow{F_{\text{gravity}}} = m\vec{g}$$

---

[6] [9]
[7] [10]

We are taught that the acceleration of Earths gravity is close to $\|\vec{g}\| = 9.8m/s^2$. Now look:[8]

$$\begin{aligned}\frac{F_{\text{gravity}}}{m_{\text{you}}} &= G\frac{m_{\text{Earth}}}{\left\|\vec{a}-\vec{b}\right\|^2} = \\ &= g = \\ &= 9.8\end{aligned}$$

$\left\|\vec{a}-\vec{b}\right\|$ is the radius of the Earth[9] (I guess the Earth isn't a sphere, but you get the point), so:

$$\begin{aligned}G\frac{m_{\text{Earth}}}{6371008.7714^2} = 9.8 \Leftrightarrow m_{\text{Earth}} = 9.8\frac{6371008.7714^2}{G} \Leftrightarrow \\ \Leftrightarrow m_{\text{Earth}} = 5.9598683 \times 10^{24}\ \text{kg}\end{aligned}$$

I believe I did the maths right, and as you can see, the Earth is… heavy.

n-Body Problem

> The problem of predicting the motion of n objects subject to gravity is known as the n-body problem.
>
> — [8]

The problem itself isn't hard to understand, nor to implement. You obviously just calculate all of the accelerations to then apply to each object given Δt.

The problem then comes when the number of objects grow, because a calculation for the acceleration of an object depends on all other objects. Time complexity $n(n-1) = \mathrm{O}(n^2)$, which doesn't look that crazy.

Literally, like it's shown in the project statement:

$$\begin{aligned}\vec{a_i} &= \frac{\vec{F_i}}{m_i} = \\ &= \frac{1}{m_i}\sum_{j=0,j\neq i}^{n}\overrightarrow{F_{ij}} = \\ &= \frac{Gm_i}{m_i}\sum_{j=0,j\neq i}^{n}\frac{m_j\overrightarrow{\Delta r}}{\left\|\overrightarrow{\Delta r}\right\|^3} = \\ &= G\sum_{j=0,j\neq i}^{n}\frac{m_j\overrightarrow{\Delta r}}{\left\|\overrightarrow{\Delta r}\right\|^3}\end{aligned}$$

The project statement actually introduces another variable, ε, which prevents from dividing by zero in case the two objects are in the exact same position. I guess that's not even possible in real life since there's flesh around my centre of gravity that prevents us from being in the same position, I guess that's unless something is part of me. To mimic real life, Each of the objects for the simulation will get a "*radius*" greater than zero, representing the "flesh", the collision box. This will serve exactly like the *softening parameter* ε:

---

[8] [9]
[9] [11]

$$\because \text{radius}_a > 0 \land \text{radius}_b > 0$$
$$\varepsilon := \text{radius}_a + \text{radius}_b$$
$$\therefore \varepsilon > 0$$
$$\therefore \forall x \in \mathbb{R}, \left\|\overrightarrow{\Delta r}\right\|^3 + \varepsilon > 0$$
$$\therefore \forall x \in \mathbb{R}, \left\|\overrightarrow{\Delta r}\right\|^3 + \varepsilon \neq 0$$

Division by zero won't happen this way: $\varepsilon > 0 \Rightarrow \forall x \in \mathbb{R}, \left\|\overrightarrow{\Delta r}\right\|^3 + \varepsilon \neq 0$.

Later, this idea of collisions can later serve the purpose to actually physically simulate them so that it becomes impossible for two or more objects to be "on top" of each other.

But now, where in the formula of acceleration would I put this? I think I just don't put it, and collisions should be taken care of programmatically and not with maths.

I might have just not understood fully how ε integrates on the formula, because, why are we doing the Euclidean distance between $\|\Delta r\|$ and ε, to the power of 3, to substitute $\|\Delta r\|$ alone, and what's the logic behind it? Are there other solutions for the obvious problem? And also, is there an optimal value for ε then? I really don't get it.

I got it that we need it to "remove the singularity at small distances"[10], but that's it. And by talking about optimal values for ε, I haven't even thought about the values for the other parameters. Maybe I'll just consider $G = 1$ to simplify things, as I don't think I'll be simulating the cosmos.

What a *dilemma*!

The singularity dilemma

$$\vec{a_i} = G \sum_{j=0, j\neq i}^{n} \frac{m_j \overrightarrow{\Delta r}}{\left\|\overrightarrow{\Delta r}\right\|^3}$$

When $\overrightarrow{\Delta r} \to \vec{0}$:

$$\begin{aligned}
\lim_{\overrightarrow{\Delta r}\to\vec{0}} \vec{a_i} &= G \sum_{j=0, j\neq i}^{n} \left(\lim_{\overrightarrow{\Delta r}\to\overrightarrow{0^+}} \frac{m_j}{\left\|\overrightarrow{\Delta r}\right\|^2} \frac{\overrightarrow{\Delta r}}{\left\|\overrightarrow{\Delta r}\right\|} \right) \vee G \sum_{j=0, j\neq i}^{n} \left(\lim_{\overrightarrow{\Delta r}\to\overrightarrow{0^-}} \frac{m_j}{\left\|\overrightarrow{\Delta r}\right\|^2} \frac{\overrightarrow{\Delta r}}{\left\|\overrightarrow{\Delta r}\right\|} \right) = \\
&= G \sum_{j=0, j\neq i}^{n} \frac{\pm\infty \overrightarrow{\Delta r}}{\left\|\overrightarrow{\Delta r}\right\|} \\
&= \pm\vec{\infty}
\end{aligned}$$

I mean, you can imagine that because the closer two objects are to each other, the stronger the gravitational force is, there becomes a point that they are so close together that that force is infinitely big. I just can't fathom what happens to the direction of that vector in this case.

So that's another problem I did not think of earlier. We should "cap" huge speeds, but do we just reuse the last trajectory?

The project statement replaces $\left\|\overrightarrow{\Delta r}\right\|$ with $\sqrt{\left\|\overrightarrow{\Delta r}\right\|^2 + \varepsilon^2}$. The logic is that there is always a *distance* that ε can "simulate" to be lesser than — but not 0, as that would destroy the purpose of it — that makes this substitution always greater than 0 and still be able to make $\left\|\overrightarrow{\Delta r}\right\| \approx \sqrt{\left\|\overrightarrow{\Delta r}\right\|^2 + \varepsilon^2}$ so

[10] [12]

that the acceleration calculated is close to reality, which only happens when ε is way, way smaller than $\left\|\overrightarrow{\Delta r}\right\|$:

$$\because r := \left\|\overrightarrow{\Delta r}\right\|, r \geq 0, \forall \varepsilon \in \mathbb{R}, \sqrt{r^2 + \varepsilon^2} = 0 \Leftrightarrow r = \varepsilon = 0$$

$$\therefore \forall \zeta > 0, \exists \delta = \delta(\zeta, r), \delta > 0, \forall \varepsilon \in \mathbb{R}, 0 < |\varepsilon| < \delta \rightarrow \left|\sqrt{r^2 + \varepsilon^2} - r\right| < \zeta$$

Here, δ represents the distance that ε can't be greater than, and δ depends on all the real distances between all objects $\left\|\overrightarrow{\Delta r_i}\right\|$[11], but also on how much we want the gravitational force calculation to be close to reality, represented by ζ. So if we want ζ accuracy/precision/tolerance in our physics, $\therefore |\varepsilon| < \delta\left(\zeta, \left\{x : \left\|\overrightarrow{\Delta r_x}\right\|\right\}\right)$. The gravitational force stops being accurate to what would happen in reality once $\left\|\overrightarrow{\Delta r}\right\| < |\varepsilon|$, which can only be an accurate value, again, only if $|\varepsilon|$ is way, way smaller than $\left\|\overrightarrow{\Delta r}\right\|$.

Now we do not have the problem of dividing by 0 any more, but we have an idea of what the value of ε is based on (we think), but still don't know how to exactly calculate it. It's about when we stop caring about the real physics that are being applied, but we don't know what number we would like it to be for each possible universe of objects and their states. We might just start by making it a static number $10^{-x}, x \geq 1$, just so it doesn't affect the calculations a lot but still prevents singularity, see what happens, and tweak it from there.

——— * ⁂ * ———

Implementation

Main loop

I knew I wanted some sort of "professional game loop", something I had an idea on how to go about it.

Still, I wanted to make something a little different from the usual. I wanted to maybe have multiple simulations running in parallel — threads allow that.

I also wanted an API that resembles a media player, with `start`, `pause` and `stop` "buttons", "stepping forward" on a paused simulation (so we can control "how much time is passing"). We can pause a simulation and look at what is happening in it.

Wasn't sure about what's the best way to go about managing the Δt generation. The solution was on using the Strategy design pattern.

So I implemented the `Loop` class, which also is an iterator and a context manager, so we can use it simply like this:

```
with Loop(lambda: fixed_step(1/24)) as loop:
    loop.skip_forward(3)
    for dt in loop:
        print(dt)
```

One decision I had to make is the choosing of the function of the `time` module that we would use to calculate the time differences. There were various options; `time.monotonic` seemed like the way to go. But I didn't look too much into it and still didn't have the opportunity to test different strategies.

[11] The thing is that, distances between objects change, but I don't know if ε is supposed to change or just to be a constant from the start to the end of the simulation.

Velocity Verlet

I tried to understand why we do it that way by reading the Wikipedia page, but I didn't exactly. So I just tried to implement it by looking at the project statement.

My first implementation was a translation to code to what I was seeing:

```
from nbodysim.object import Object, Vec
from . import Backend, Env
from nbodysim import G
import numpy as np

def step(self, dt: float, env: Env) -> Env:
  updated = []

  for i in range(len(env.objects)):
    x, a = env.objects[i]

    x.position += x.velocity * dt + (1 / 2) * a * dt**2
    x.velocity += (1 / 2) * a * dt

    def accelaration(y: Object) -> Vec:
      diff = y.position - x.position
      return (y.mass * diff) / ( (np.linalg.norm(diff) ** 2 + env.epsilon**2) ** (3 /
2) )

    a = np.zeros_like(x.position)
    for j, (y, _) in enumerate(env.objects):
      if j == i:
        continue

      a += accelaration(y)
    a *= G

    x.velocity += (1 / 2) * a * dt

    updated.append((x, a))

  env.objects = updated
  return env
```

Then I noticed that, by not updating all positions and velocities all at once (right now we are doing one by one with the iterator), it's like it actually passed $n * \Delta t$ time, and only one object updates for each Δt, already influencing the others' acceleration. They should update all at the same time.

A solution is to simply save all initial positions (actually the current objects state) in an array like `python objects = [o for o, _ in env.objects]`, and use that specifically for calculations.

NumPy

Finnally, I learned how to actually use the `numpy` module, not just to store values. I started by reading the documentation.

After that, I still didn't understand how should I be using it in this situation about the acceleration. My idea was that I was not supposed to be using loops for the calculations.

```
def step(self, dt: float, env: Env) -> Env:
    objs = [obj for obj, _ in env.objects]
    a = np.array([a for _, a in env.objects])
```

```
    r = np.stack([o.position for o in objs])
    v = np.stack([o.velocity for o in objs])
    m = np.array([o.mass for o in objs])

    r += v * dt + (1 / 2) * a * dt**2
    v += (1 / 2) * a * dt

    # TODO: calculate accelaration
    # a = G * ...

    v += (1 / 2) * a * dt

    for i, o in enumerate(objs):
        o.position = r[i]
        o.velocity = v[i]

    env.objects = list(zip(objs, a))
    return env
```

With the help of generative AI, I found out how to go about it using matrix multiplication and manipulation, specially by increasing the dimensions of certain arrays/matrixes.

Were is an example, where we take a `[N, dim]` array of vectors of the position (`N` is the number of objects and `dim` the dimensions the simulation will be working on) and create a `[N, N, dim]` where each position is the difference of the positions of two objects:

```
diff = r[np.newaxis, :, :] - r[:, np.newaxis, :]
```

I tried commenting this part of the code so it becomes easier to understand what is happening.

CUDA

Using the example code given by the project statement, I did figure out what to do in the CUDA kernel pretty easy. But I got a little stuck on how exactly to use PyCuda now, specially on how to pass, my data, from host to device *et vice versa*. Generative AI did guide me:

- I need to `raven`/flatten the arrays so that CUDA can read them with copy;
- `float32` is way faster than `float64`,

Simulate

With all this components made, I just have to make them work together. I created the `Simulator` class that works with both the `Loop` and `Backend` at the same time — passing the `Loops` generated differences of time to the `Backend` to use it to update our environment `Env`.

I also did a way to create an `Env` randomly. But I don't think I'm doing that right. The initial values don't seem to be good for the size of our simulations.

At first, objects had an initial velocity, but the objects were actually getting apart from each other. My idea is that the mass is to small to create gravitational force, because I mean, it's multiplied by the big G. Then I jumped to make all initial velocities equal to zero to see what happens, and it's like the objects are not moving at all. But accelerations existed — I checked. They are just really tiny accelerations. That's really why me and you, human reader with a similar weight as me, don't go towards each other even if we are very close together.

CLI
To create a simulation, I decided to create an CLI that let's use give some parameters on how we want the simulation to go from the 3 parts: Selection of `Loop` strategy and Backend, and how to create the initial environment.

Matplotlib
As I said, some sections above, I was able to visualize that our initial data is pretty bad, but that wasn't only by using `python print()` statements. For now, we did a 2 dimensions scatter to visualize our environment being simulated.

Conclusion
You can check out https://github.com/Marado-Programmer/nbodysim for the Git repository!

This project explored the implementation of an n-body simulation using both CPU and GPU approaches, focusing on numerical stability, performance, and parallel execution. While the physics model is conceptually simple, achieving stable and realistic behavior required careful attention to integration methods, parameter scaling, and computational structure.

Appendices

Dependencies and Modules discovery
I'm not very familiar with the Python ecosystem.

I already knew about creating a python virtual environment to then install the packages with `pip`, create the `requirements.txt`. But when I started searching what a common project structure for Python looks like, I learned about the `pyproject.toml` file standard. Then I don't know exactly how it came to be, but I ended up in Scientific Python which introduced me to Pixi which I decided to try out.

One website that I always come back to when I want to view the basics of a programming language, skill, etc., is https://roadmap.sh/. I went there to find what were the standard/builtin modules that Python provided and that could be important for me to know. Ended up in Python's complete documentation.

Modules
The module `threading` allows for parallelism using threads. It's used to allow multiple `Loop`s to run simultaneously, meaning that we can have multiple simulations running at the same time.

We also get to use quite a lot the `threading.Event` class that allows as to control the `Loop` with a media player controllers-like API.

We use a queue data structure for a FIFO way of storing the differences of time (implemented by the `queue` module).

The `time` module lets us calculate the differences of time.

Between `argparse` and `optparse`, the first seemed the faster to use and create an CLI.

Bibliography

[1] "Wikipedia, the Free Encyclopedia." Accessed: Apr. 13, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Main_Page

[2] "Crash Course Physics (YouTube Playlist)." Accessed: Apr. 13, 2026. [Online]. Available: https://www.youtube.com/playlist?list=PL8dPuuaLjXtN0ge7yDk_UA0ldZJdhwkoV

[3] “Motion in a Straight Line: Crash Course Physics #1 (Video).” Accessed: Apr. 13, 2026. [Online]. Available: https://www.youtube.com/watch?v=ZM8ECpBuQYE

[4] “Velocity.” Accessed: Apr. 13, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Velocity

[5] “Derivatives: Crash Course Physics #2 (Video).” Accessed: Apr. 13, 2026. [Online]. Available: https://www.youtube.com/watch?v=ObHJJYvu3RE

[6] “Integrals: Crash Course Physics #3 (Video).” Accessed: Apr. 13, 2026. [Online]. Available: https://www.youtube.com/watch?v=ObHJJYvu3RE

[7] “Motion in a Straight Line: Crash Course Physics #1,” Physics. Accessed: Apr. 13, 2026. [Online]. Available: https://thecrashcourse.com/courses/motion-in-a-straight-line-crash-course-physics-1/

[8] “Newton's Law of Universal Gravitation.” Accessed: Apr. 13, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Newton%27s_law_of_universal_gravitation

[9] “Newtonian Gravity: Crash Course Physics #8 (Video).” Accessed: Apr. 13, 2026. [Online]. Available: https://www.youtube.com/watch?v=7gf6YpdvtE0

[10] “Gravitational Constant.” Accessed: Apr. 13, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Gravitational_constant

[11] “Earth Radius.” Accessed: Apr. 13, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Earth_radius

[12] “N-body Problem.” Accessed: Apr. 13, 2026. [Online]. Available: https://en.wikipedia.org/wiki/N-body_problem